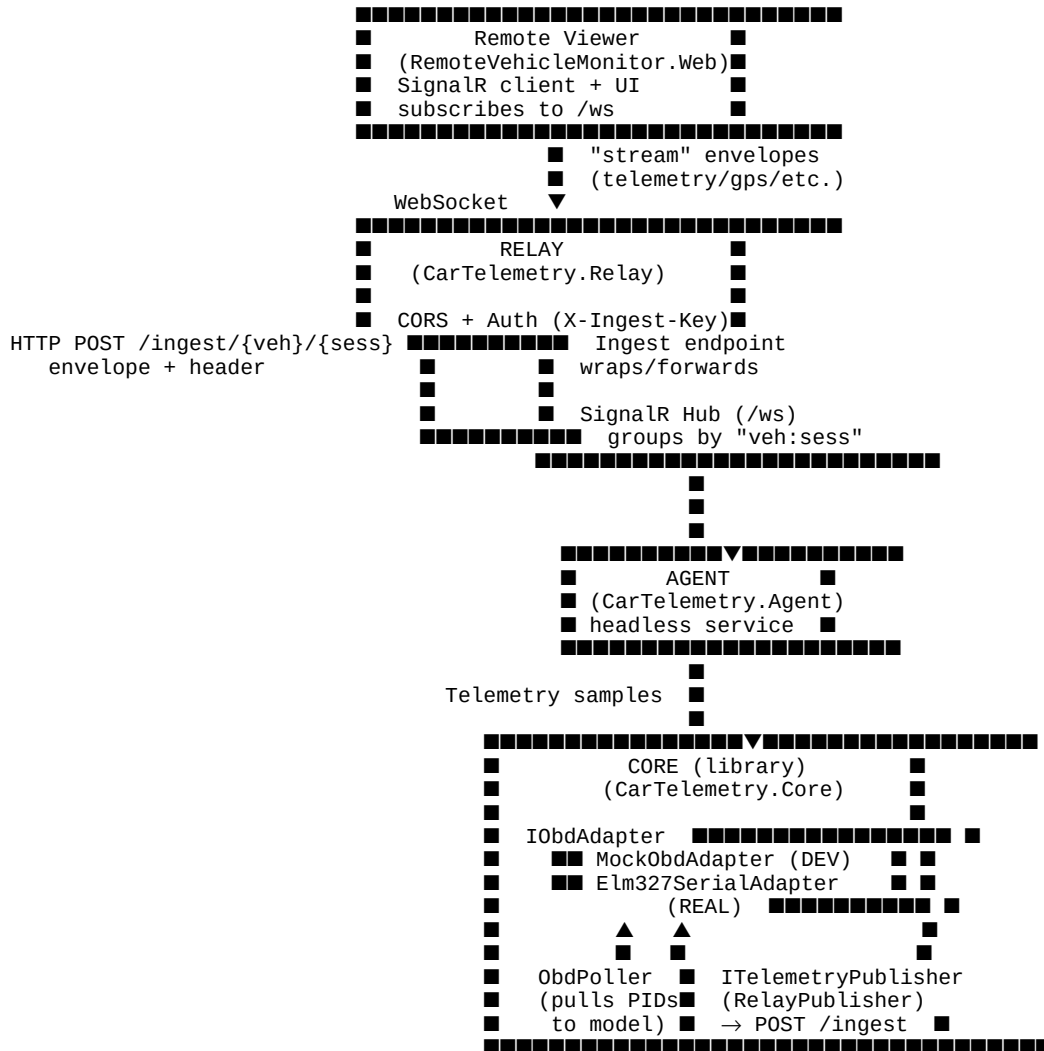


CarTelemetry System Flowchart and Architecture Overview



CarTelemetry.Core (library)

- `IObdAdapter` = hardware abstraction
- `MockObdAdapter` → DEV source (fake OBD samples)
- `Elm327SerialAdapter` → REAL source (serial ELM327; reads PIDs)
- `ObdPoller` → uses the adapter to produce Telemetry data
- `ITelemetryPublisher` (`RelayPublisher`) → wraps each sample in an envelope and sends to the relay
- `Telemetry` model → { Rpm, SpeedKmh, CoolantC, TsUtcMs }

CarTelemetry.Agent (headless on Pi/PC)

- DI wires adapters and publisher
- `TelemetryPumpService` loops: poll → publish
- DEV: Mock adapter; REAL: Elm327 adapter

CarTelemetry.Relay (server)

- `/ingest/{veh}/{sess}` endpoint accepts data with header `X-Ingest-Key`
- Broadcasts data via SignalR hub `/ws` grouped by `veh:sess`
- Uses CORS to allow viewer origin

RemoteVehicleMonitor.Web (viewer)

- SignalR client connects to relay
- Displays incoming telemetry stream
- Future: add GPS map, replay, analytics

CarTelemetry.Desktop (in-car GUI)

- Local GUI for in-car display
- Uses ObdPoller directly (mock for now)
- Doesn't depend on relay

Swap for Real Hardware: In *CarTelemetry.Agent/Program.cs* change adapter binding: DEV (today): `services.AddSingleton(); REAL (ELM327): const string PortName = "COM3"; // Windows // const string PortName = "/dev/ttyUSB0"; // Raspberry Pi/Linux const int Baud = 38400; services.AddSingleton(_ => new Elm327SerialAdapter(PortName, Baud));`
Run Order:

- 1) Relay
`dotnet run --project .\CarTelemetry.Relay`
- 2) Agent
`dotnet run --project .\CarTelemetry.Agent`
- 3) Viewer
`dotnet run --project .\RemoteVehicleMonitor.Web`

Set relay/vehicle/session in viewer and Save.